Step 2 — Saved State and a Human Pause Before Anything Goes Out — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: a graph can do something a hand-written loop can't: **stop in the middle, save everything, and continue later** — even after the program was closed. That's what lets a person approve an answer before it goes out. This step adds the Approval agent's file and node — `approval_node`, which for now asks a person about **every** answer — plus a checkpointer and a thread id per run. **If not:** Step 5 would have to build the pause, the saved state and the node at the same time as its checks.

After this step every answer waits for your yes/no, and a paused run survives a restart. `answer_node` and `clean_task` don't change. Files not shown here don't change.

Solution — the Approval agent's node, a SQLite checkpointer, and a real restart test

Changed from Step 1: `state.py` adds `approved`.

```
# src/documind/state.py
import operator
from typing import Annotated, TypedDict

class DocState(TypedDict):
    # why: one typed shape listing everything any node can read or write
    task: str
    # how: operator.add appends each node's entries instead of replacing
    # the list — so path is a running log of the nodes that ran (Doc09)
    path: Annotated[list[str], operator.add]
    # new in Step 2: whether a person (or later, the checks) approved it
    approved: bool
    answer: str

def start_state(task: str) -> DocState:
    # why: every field starts with a known value, so no node ever has to
    # guess whether a key exists yet
    return {"task": task, "path": [], "approved": False, "answer": ""}
```

Story — `agents/approval_agent.py`: the Approval agent's file, with its node from day one. `approval_node` calls `interrupt()`, which pauses the graph and saves the state; whatever you pass to `Command(resume=...)` comes back as `decision` (Doc09). For now its list of `reasons` is one fixed line, so **every** answer goes to a person — Step 5 replaces that one line with real checks. (Also create an empty `agents/__init__.py`.) **If not:** answers would go out with no human check at all.

```
# src/documind/agents/approval_agent.py
from langgraph.types import interrupt

from documind.state import DocState
from documind.utils.logger import get_logger

def approval_node(state: DocState) -> dict:
    logger = get_logger(__name__)
    # why: for now EVERY answer is checked by a person — Step 5 replaces
    # this one line with real checks, and approves safe answers alone
    reasons = ["every answer is checked by a person for now"]

    logger.info("asking a human, failed checks: %s", reasons)
    # how: interrupt() pauses the graph here and saves the state; the
    # value passed to Command(resume=...) comes back as `decision`
    # when: on resume, this node runs again from its first line — so the
    # log line above prints twice, and nothing above interrupt() may do a
    # real action (like sending an email)
    decision = interrupt({"answer": state["answer"], "failed_checks": reasons})
    if decision == "yes":
        return {"approved": True, "path": ["human_approved"]}
    return {"approved": False,
            "answer": "Not approved — nothing was sent.",
            "path": ["human_rejected"]}
```

Changed from Step 1: a third node, and `build_graph()` now takes a checkpointer.

Story — `graph.py`: the checkpointer is **passed in** (Doc09's Build Task): `main.py` passes a SQLite one that writes to disk, tests pass a `MemorySaver`. **If not:** the checkpointer would be hard-coded, and a test and the app couldn't each use their own.

```
# src/documind/graph.py
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

from documind.agents.approval_agent import approval_node
from documind.nodes import answer_node, clean_task
from documind.state import DocState
```

```
def build_graph(checkpointer=None):
    # why: the checkpointer is passed in (Doc09) — main.py uses a SQLite
    # one that survives a restart, the tests use a fresh MemorySaver
    if checkpointer is None:
        checkpointer = MemorySaver()

    builder = StateGraph(DocState)
    builder.add_node("clean_task", clean_task)
    builder.add_node("answer_node", answer_node)
    builder.add_node("approval_node", approval_node)

    builder.add_edge(START, "clean_task")
    builder.add_edge("clean_task", "answer_node")
    builder.add_edge("answer_node", "approval_node")
    builder.add_edge("approval_node", END)
    # how: without a checkpointer, interrupt() has nowhere to save state
    return builder.compile(checkpointer=checkpointer)
```

Changed from Step 1: `main.py` opens a `SqliteSaver`, gives each question its own thread id, and handles the pause with `ask_human()`.

Story — `main.py`: `SqliteSaver.from_conn_string(DB_PATH)` stores every checkpoint in `checkpoints.sqlite` (add it to `.gitignore`). Each question gets its own `thread_id`, so runs never mix. When the result contains `"__interrupt__"`, `ask_human()` shows the answer and the reason, asks yes/no, and the app resumes with `Command(resume=decision)`. **If not:** a paused run would be lost on restart, and two questions could share one saved state.

```
# src/documind/main.py
import sys
import time

import openai
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command

from documind.config import load_config
from documind.exceptions import MissingConfigError
from documind.graph import build_graph
from documind.state import start_state
from documind.utils.logger import get_logger

# why: a file on disk — a paused run survives even if the app is closed
DB_PATH = "checkpoints.sqlite"

def ask_human(request: dict) -> str:
```

```
    print("  -- needs your approval --")
    print(f"  answer: {request['answer']}")
    print(f"  why:   {', '.join(request['failed_checks'])}")
    return input("  Approve? (yes/no): ").strip().lower()

def main() -> None:
    # why: config first — a missing OPENAI_API_KEY stops the app here
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    print("DocuMind — ask a question, or /quit to exit.")

    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        run_number = 0
        while True:
            task = input("You: ").strip()
            if task == "/quit":
                break
            if task == "":
                continue

            run_number = run_number + 1
            # why: one thread id per question — the saved state of each
            # run is kept apart; the time makes it unique across restarts
            thread_id = f"run-{int(time.time())}-{run_number}"
            config = {"configurable": {"thread_id": thread_id}}
            try:
                result = graph.invoke(start_state(task), config)
                if "__interrupt__" in result:
                    request = result["__interrupt__"][0].value
                    decision = ask_human(request)
                    result = graph.invoke(Command(resume=decision), config)
            except openai.AuthenticationError:
                logger.error("Authentication failed — check OPENAI_API_KEY.")
                print("Login failed — check OPENAI_API_KEY in your .env file.")
                sys.exit(1)
            except openai.RateLimitError:
                print("We're busy right now — wait a moment and try again.")
                continue

            print(f"  path: {' -> '.join(result['path'])}")
            print(f"Answer: {result['answer']}")
```

```
if __name__ == "__main__":
    main()
```

Story — `scripts/pause_demo.py` + `scripts/resume_demo.py`: the real proof. Part 1 starts a run and stops at the pause, then exits. Part 2 is a **separate process**: it finds the paused run in the SQLite file with `graph.get_state(config)` and finishes it. (Doc09 showed `MemorySaver` fails exactly this test.) **If not:** "it resumes" might only mean "the objects were still in memory".

```
# scripts/pause_demo.py
# Part 1 of 2: starts a run that needs approval, and stops at the pause.
# Run from the repo root: python scripts/pause_demo.py
# Then run scripts/resume_demo.py — a separate process — to finish it.
from langgraph.checkpoint.sqlite import SqliteSaver

from documind.config import load_config
from documind.graph import build_graph
from documind.state import start_state

DB_PATH = "checkpoints.sqlite"
THREAD_ID = "demo-1"
TASK = "What is the refund policy? Send it to the customer."

def main() -> None:
    load_config()
    config = {"configurable": {"thread_id": THREAD_ID}}
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        result = graph.invoke(start_state(TASK), config)
    if "__interrupt__" in result:
        request = result["__interrupt__"][0].value
        print(f"Paused on thread {THREAD_ID}, waiting for approval:")
        print(f"  answer: {request['answer']}")
        print("Now close this and run: python scripts/resume_demo.py")
    else:
        print("The run finished without pausing — nothing to resume.")

if __name__ == "__main__":
    main()
```

```
# scripts/resume_demo.py
# Part 2 of 2: a NEW process that picks up the paused run from disk.
# Run from the repo root, after scripts/pause_demo.py.
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import Command
```

```
from documind.config import load_config
from documind.graph import build_graph

DB_PATH = "checkpoints.sqlite"
THREAD_ID = "demo-1"

def main() -> None:
    load_config()
    config = {"configurable": {"thread_id": THREAD_ID}}
    with SqliteSaver.from_conn_string(DB_PATH) as checkpointer:
        graph = build_graph(checkpointer)
        # why: read the saved state first — this process never saw the
        # run start, so everything it knows comes from the checkpoint file
        saved = graph.get_state(config)
        if not saved.next:
            print(f"Nothing is paused on thread {THREAD_ID}.")
            return
        print(f"Found a paused run: {saved.values['task']}")
        decision = input("Approve? (yes/no): ").strip().lower()
        result = graph.invoke(Command(resume=decision), config)
    print(f"path: {' -> '.join(result['path'])}")
    print(f"Answer: {result['answer']}")

if __name__ == "__main__":
    main()
```

Run it:

```
python scripts/pause_demo.py      # pauses, then exits
python scripts/resume_demo.py     # a new process finishes the run
python -m documind.main
```

Expected output (the answer's wording varies):

```
$ python scripts/pause_demo.py
Paused on thread demo-1, waiting for approval:
  answer: Refund policies usually allow returns within 14-30 days ...
Now close this and run: python scripts/resume_demo.py

$ python scripts/resume_demo.py
Found a paused run: What is the refund policy? Send it to the customer.
Approve? (yes/no): yes
path: clean_task -> answer_node -> human_approved
Answer: Refund policies usually allow returns within 14-30 days ...
```

Notice the answer is a *general* guess — the graph can't read your documents yet. That's Step 3.